

Personal AI Assistant — Build & Understand Prompts (Antigravity CLI)

How to use this doc: for each phase, run the **Build** prompt(s) first. Once Antigravity finishes, run the **Understand** prompts *in the same session* (so it has context on what it just built) before touching Phase N+1. Don't move on until you can answer the **Checkpoint Questions** yourself, without asking the agent.

Phase 1 — Foundation (Project setup, Auth, Chat UI, Basic API)

Build prompts

```
Set up a Next.js 14 (App Router) + TypeScript + Tailwind frontend and a FastAPI backend in a monorepo structure: /apps/web, /apps/api, /packages/shared. Add NextAuth.js for email/password auth on the frontend, matching JWT auth on the FastAPI backend. Add a basic PostgreSQL connection via SQLAlchemy with a `users` table. Include a working dev script that runs both frontend and backend concurrently.
```

```
Build a chat UI (single conversation, no persistence yet) that streams responses from a FastAPI endpoint using the Vercel AI SDK. The FastAPI endpoint should call an LLM provider (Anthropic) and stream tokens back via SSE. Keep state management with Zustand.
```

Understand prompts (ask Antigravity)

```
Walk me through every file you just created or changed, in the order a request would flow through them: browser → Next.js → FastAPI → LLM → back to browser. Explain what each file is responsible for.
```

Explain how the JWT auth flow works end to end: what happens when I log in, where the token is stored, and how a protected API route verifies it. What would break if I deleted the middleware?

Explain how streaming works here – what is SSE, why did we use it instead of a normal JSON response, and what's the exact code path that turns LLM tokens into what I see typing on screen?

What are the security weaknesses in what you just built? What would you NOT ship to production as-is?

Checkpoint questions (answer yourself, no agent)

- Can I draw the request path from browser click to LLM response on a whiteboard?
 - Where exactly does the JWT get validated, and what happens if it's expired?
 - If I wanted to swap Anthropic for OpenAI, which files would I touch?
-

Phase 2 — Memory (Embeddings, storage, retrieval, use in chat)

Build prompts

Add a `memories` table with a pgvector column. Create a background process that, after each chat message, decides whether it contains information worth remembering (e.g. preferences, facts about the user) and if so, embeds it and stores it with metadata. Use OpenAI or Anthropic's embedding model – recommend the best option and explain why.

Add a retrieval step to the chat endpoint: before calling the LLM, do a similarity search against `memories` for the current user, and inject the top N relevant results into the system prompt. Show me the exact prompt structure you're using.

Understand prompts

Explain your reasoning for what counts as "worth remembering" in the code you wrote.
Show me 3 example messages that WOULD get stored and 3 that WOULDN'T, and why.

Walk me through vector similarity search step by step: what does the embedding actually represent, what distance metric are we using (cosine? L2?), and why does a smaller/larger distance mean more or less similar?

What happens if two memories contradict each other (e.g. user says they're vegetarian, then later says they love steak)? Does the current implementation handle that, or would both memories get retrieved and confuse the model? What would you change?

What are the failure modes of this memory system at scale – 10,000 memories for one user? Would retrieval quality degrade, and how would we fix that later?

Checkpoint questions

- Can I explain embeddings to a non-technical friend using an analogy?
 - Do I know why we chose the embedding model we chose (cost vs. dimensionality vs. quality)?
 - Can I identify, in the code, exactly where retrieved memories get inserted into the prompt?
-

Phase 3 — Files & RAG (Upload, extract, chunk, embed, answer from files)

Build prompts

Add file upload (PDF, TXT, MD) to Supabase Storage, with a `files` table tracking metadata. Build an extraction pipeline: pull text out of each file type, chunk it

(explain your chunking strategy and why), embed each chunk, and store it in a `file\_embeddings` pgvector table linked to the file and user.

Add a "chat with your files" mode: when the user references an uploaded file or asks a question, retrieve relevant chunks via similarity search and include them in the LLM context with citations back to the source file.

Understand prompts

Explain your chunking strategy in detail: chunk size, overlap, and why. What would go wrong if I set chunk size way too small (e.g. 50 tokens) or way too large (e.g. 8000 tokens)?

Show me one real chunk from a file I uploaded and its embedding's nearest neighbors.
Do the retrieved chunks actually look relevant to a test question I ask?

Walk me through what happens if I upload a 200-page PDF – where's the bottleneck, and would this current implementation time out on Vercel's serverless functions? If so, what's the fix?

What's the difference between what we built (RAG) and just pasting the whole document into the context window? When would you use one over the other?

Checkpoint questions

- Can I explain, unprompted, why RAG exists instead of just always dumping full documents into context?
 - Do I know what "chunk overlap" is and why it matters?
 - Can I manually trace a bad answer back to a bad retrieval, if one happens?
-

Phase 4 — Tools (Web search, vision, code assistant)

Build prompts

Add tool calling to the chat endpoint using the Anthropic tool use API: implement a web_search tool (via Tavily or Serper) and wire it so the model can decide when to call it. Show me the full tool definition schema and explain each field.

Add a vision endpoint: user uploads or references an image, and the model can analyze it and answer questions. Add a code assistant mode that can explain, generate, and debug code snippets in the chat.

Understand prompts

Explain tool use step by step: how does the model "decide" to call a tool, what gets sent back to it after the tool runs, and how does it turn a tool result into a final answer? Show me a real trace from one request.

What happens if the web search tool fails or times out – does the whole chat request fail, or does the model gracefully continue without it? Show me where that's handled in the code, or tell me if it isn't.

Why did we structure the tool schema the way we did? What would happen if a required field were missing or the wrong type?

Checkpoint questions

- Can I explain tool calling to someone who's never heard of it, using this project as the example?
 - Do I know what happens on a tool failure, without looking it up?
-

Phase 5 — Productivity (Notes, Todos, Calendar, Reminders)

Build prompts

Add CRUD for notes and todos (tables, API endpoints, UI). Let the AI create/update todos and notes from natural conversation using tool calling, reusing the pattern from Phase 4.

Integrate Google Calendar API for viewing and creating events, and add a reminders system. Explain the OAuth flow needed for Calendar access.

Understand prompts

Walk me through the OAuth flow for Google Calendar end to end: what scopes are we requesting, where do tokens get stored, and how do we refresh an expired token?

Show me how a natural-language request like "remind me to call mom tomorrow at 5pm" becomes a structured database row. Where does date/time parsing happen, and what edge cases could break it (timezones, ambiguous phrasing)?

What's reused from Phase 4's tool-calling pattern here, and what's genuinely new? Point me to the diff.

Checkpoint questions

- Can I explain OAuth scopes and token refresh without looking at docs?
 - Do I understand exactly how natural language becomes a calendar event in the database?
-

Phase 6 — Polish (UI improvements, error handling, performance, deploy)

Build prompts

Audit the whole app for error handling gaps — API failures, empty states, loading states — and fix them. Add rate limiting and basic input validation across endpoints.

Set up deployment: frontend to Vercel, backend to Railway or Render, database on Supabase, with environment variables documented. Add a basic CI check (lint + type check) on push.

Understand prompts

List every place you added error handling and explain the failure scenario each one guards against. Are there any gaps left?

Walk me through the full deployment pipeline: what happens from `git push` to the app being live? What could go wrong at each step?

What's our current biggest performance bottleneck, and why? What would you profile first if the app felt slow in production?

Checkpoint questions

- Could I deploy this from scratch on a fresh machine using only my own notes?
 - Can I name the top 3 things that would break first if 100 people used this at once?
-

General habit for every phase

After the Understand prompts, do one more thing yourself: **close Antigravity and try to explain the phase out loud (or write 4-5 sentences) without looking at the code.** If you can't, go back and ask a follow-up — that's the gap in your understanding, not a sign to move faster.

If you get stuck explaining something, a good follow-up template is:

Explain [specific thing] to me like I'm a junior engineer joining this project today.
Use a concrete example from the code you just wrote, not a generic explanation.